

Autonomous Warehouse Robot: Technical Report

Oct 3, 2026 · @Diksha

1. Summary

This project delivers a complete ROS 2 Humble autonomy stack for a differential-drive warehouse robot, from wheel odometry to autonomous navigation among shelving racks, validated end to end in Gazebo against the world file as ground truth.

The robot fuses wheel odometry and a gyroscope with an EKF, detects pallets, racks, boxes and people with YOLOv8-World, and builds both an occupancy map (slam_toolbox) and a persistent object map (a GTSAM iSAM2 factor graph). Nav2 then plans and drives through the 0.65 m aisles using a costmap that consumes the semantic map.

Headline results, measured in simulation with the A* + DWB configuration:

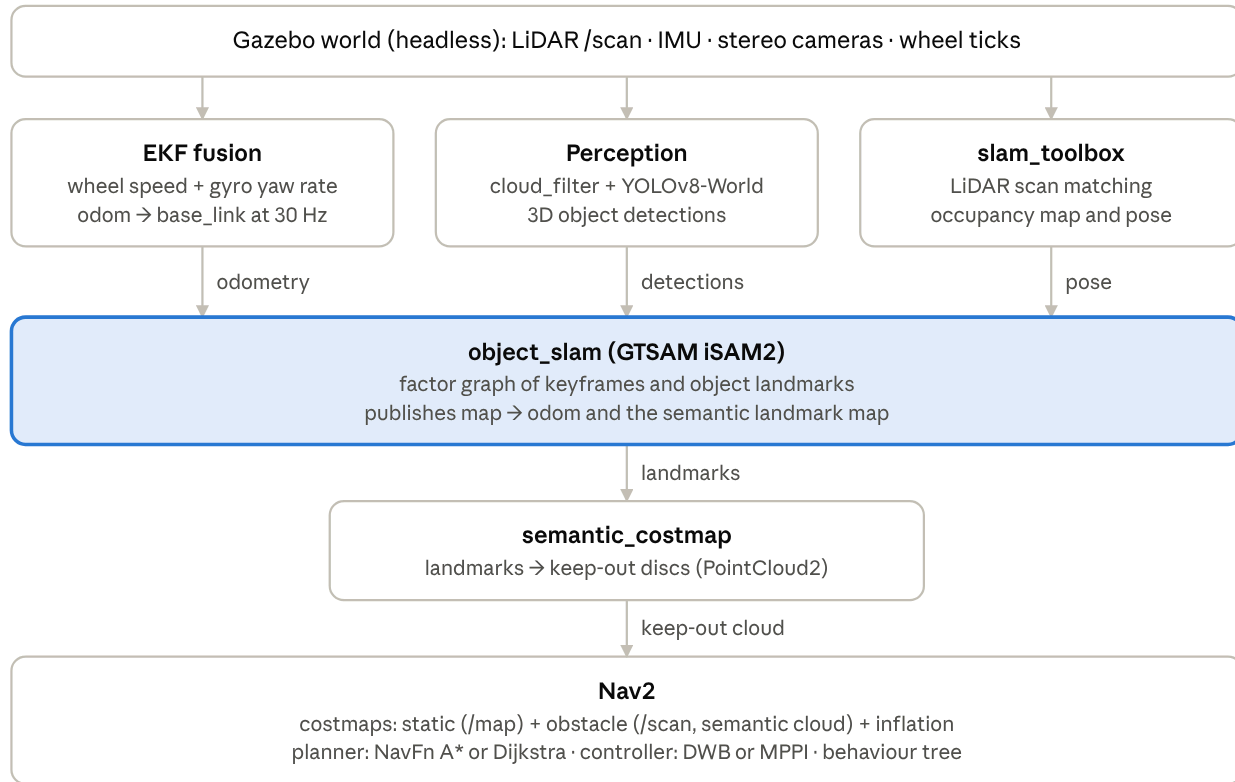
- The SLAM map recovered 84% of wall cells, 80% of rack cells and 81% of box cells at 99.9% precision. Wall recall narrowly missed its 85% target because the north wall was only 37.5% scanned.
- All 7 landmarks in the semantic map sit within 0.5 m of a real object with the correct class; 6 of 10 world objects (2 of 3 racks) were found.
- With A* + DWB, 6 of 7 mission waypoints were reached with 4.7 cm mean final error, and all 4 dynamic legs completed.
- A* and Dijkstra planned near-identical paths (3.89 m vs 3.85 m mean) in 43 ms vs 48 ms.
- Dijkstra + DWB reached 2 of 7 waypoints and A* + MPPI reached 0 of 7, so only A* + DWB navigates the whole warehouse.

Four targets were missed in the final run: wall recall (0.84 vs 0.85), mission success (0.86 vs 0.90), closest LiDAR return (0.13 m vs 0.16 m) and person separation (0.37 m vs 0.45 m). Section 9 gives the numbers and Section 10 the causes.

2. System architecture

The stack is built in layers: simulated sensors feed three estimators, object SLAM combines them into a map, and Nav2 plans on that map.

Sensors feed object SLAM, whose map Nav2 plans on



system architecture · 5 layers, sensors to Nav2

Everything runs inside one Docker image (ROS 2 Humble with Gazebo Classic, Nav2, slam_toolbox, robot_localization, PyTorch and GTSAM), with the project mounted at `/ws`. Gazebo runs headless to save CPU, and all nodes use simulated time from `/clock`. The transform chain is `map -> odom -> base_link`: the EKF owns `odom -> base_link` and object SLAM owns `map -> odom`.

3. Odometry and sensor fusion

The robot's pose estimate starts from wheel encoders and is corrected by an IMU through an Extended Kalman Filter.

Odometry (Phase 1). `wheel_tick_pub` publishes encoder ticks on `/wheel_ticks` at 50 Hz, from the simulator or a serial device. `odom_calculator` integrates the ticks into `/odom` and the `odom -> base_link` transform using differential-drive kinematics.

Sensors (Phase 2). The robot URDF carries a 2D LiDAR (`/scan`), an IMU (`/imu/data`) and a stereo camera pair, all simulated with Gazebo plugins, plus a ground-truth pose publisher used only for validation.

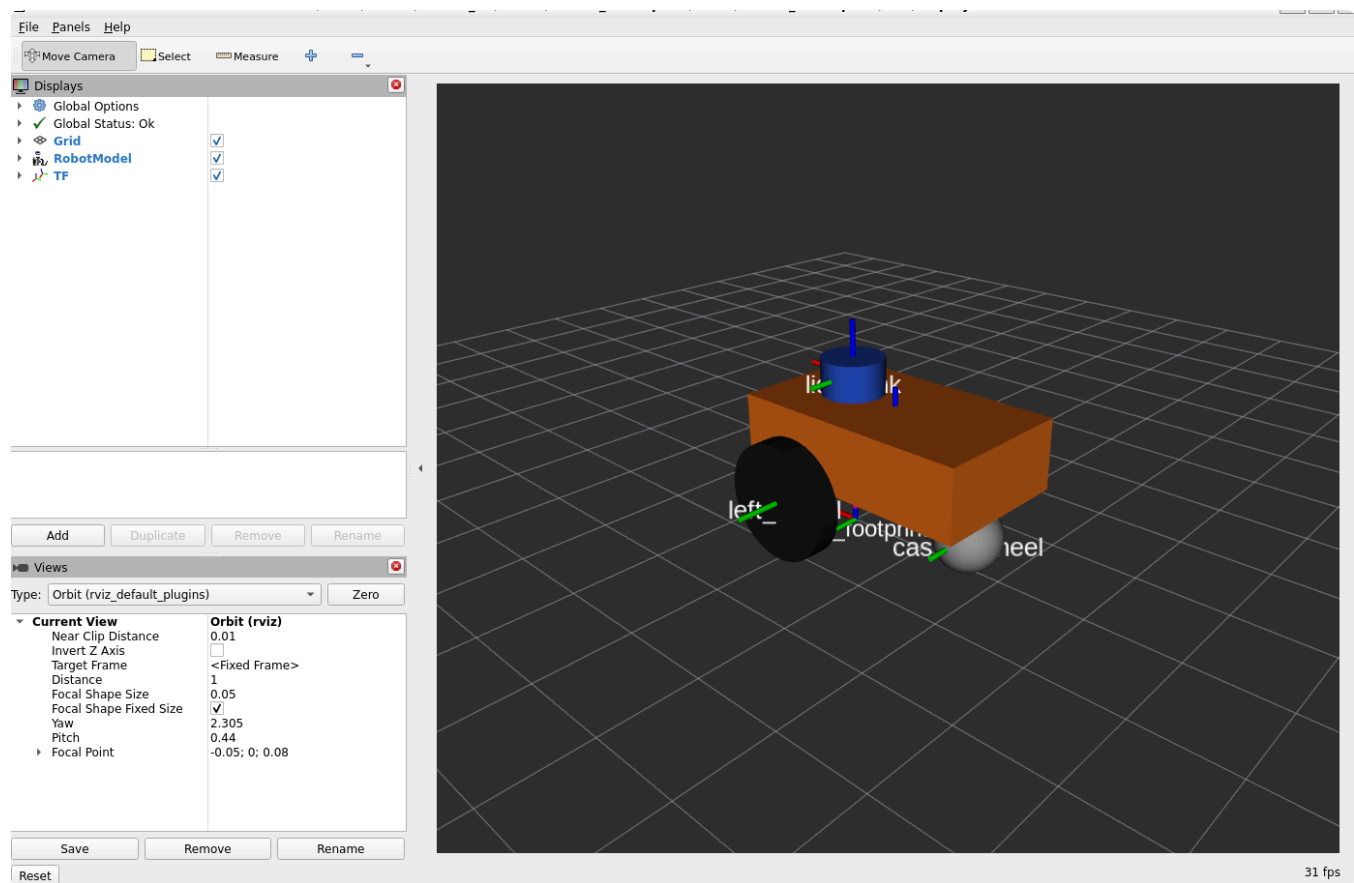


Figure: the robot model in RViz. A differential-drive base with two drive wheels and a caster, the LiDAR cylinder on top, and the TF frames of each link.

EKF. Wheels measure forward speed well but exaggerate turning when they slip; a gyroscope measures turning well but drifts if integrated alone. The `robot_localization` filter therefore takes only forward velocity (`vx`) from the wheels and only yaw rate from the IMU, runs at 30 Hz in 2D mode, and publishes the smooth `odom -> base_link` transform used by everything downstream. `tools/ekf_compare.py` compares wheel-only and fused heading against ground truth.

Task 2 package (`odom_computation`). The brief asks for a separate package that turns simulated encoder ticks into `/odom`. It is delivered as `src/odom_computation`, alongside the main stack:

Brief requirement	Implementation
Package named <code>odom_computation</code>	<code>src/odom_computation</code> (ament_python)
Subscribe to <code>/left_wheel_ticks</code> and <code>/right_wheel_ticks</code> (<code>Int32</code>)	<code>odom_node</code>

Brief requirement	Implementation
Publish <code>/odom</code> (<code>nav_msgs/Odometry</code>) at 10 Hz	Timer at 10 Hz; publishes from start-up, zeros until ticks arrive
Wheel radius 0.065 m, base 0.21 m, 512 ticks per revolution	Node parameters with those defaults
Heading converted to a quaternion; pose and twist valid	<code>yaw_to_quaternion</code> , covariances set
Bonus: <code>imu_publisher</code> on <code>/imu/data</code>	<code>imu_publisher</code> with stationary and circle modes

The kinematics live in a ROS-free module, `odom_math.py` , covered by six unit tests (straight line, spin in place, angle wrapping, unit quaternion, one-revolution distance, and 10 s of driving at 0.2 m/s). Live behaviour is checked with `ros2 topic echo /odom --once` : x increases, y stays near 0 and the orientation stays near (0, 0, 0, 1). The package is separate from the main stack's `odom_calculator` , which uses a single `/wheel_ticks` array at 50 Hz and also broadcasts the `odom -> base_link` transform.

4. Perception

Perception turns stereo images into 3D detections of the objects that matter in a warehouse: pallets, shelving racks, cardboard boxes and people.

Stereo depth. The stereo pair runs through `stereo_image_proc` to a point cloud, which `cloud_filter` cleans (range limits and noise removal) before use.

Detection. `object_detector` runs YOLOv8-World on the camera image. YOLOv8-World is open-vocabulary: a text encoder (CLIP) converts words such as "pallet" or "shelving rack" into a detector once at start-up, so no custom training set was needed. Each 2D box is combined with the point cloud to give a 3D position in `base_link` , published on `/objects/detections` .

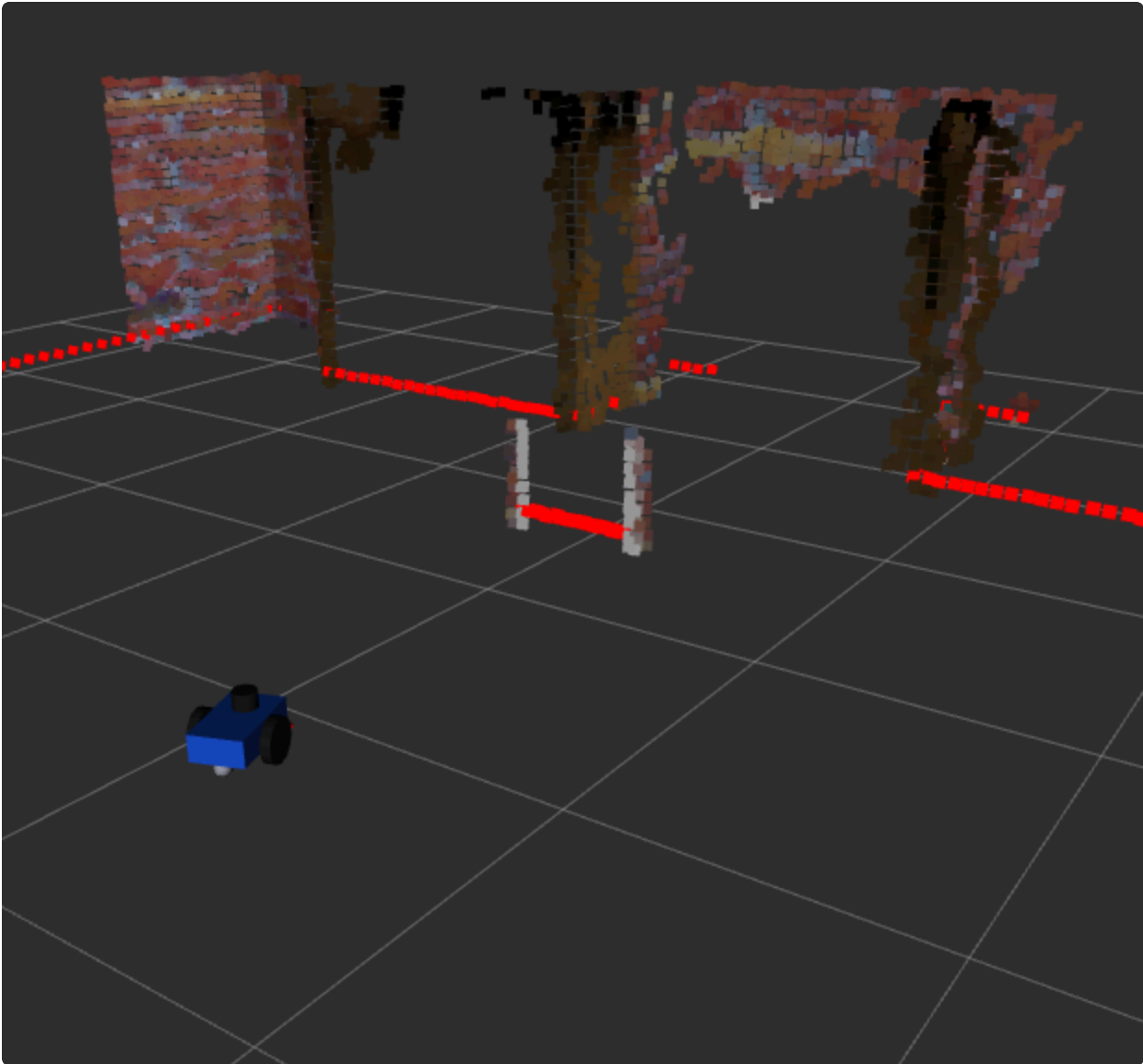


Figure: the stereo point cloud (coloured) beside the red LiDAR scan points, seen from behind the robot in RViz.

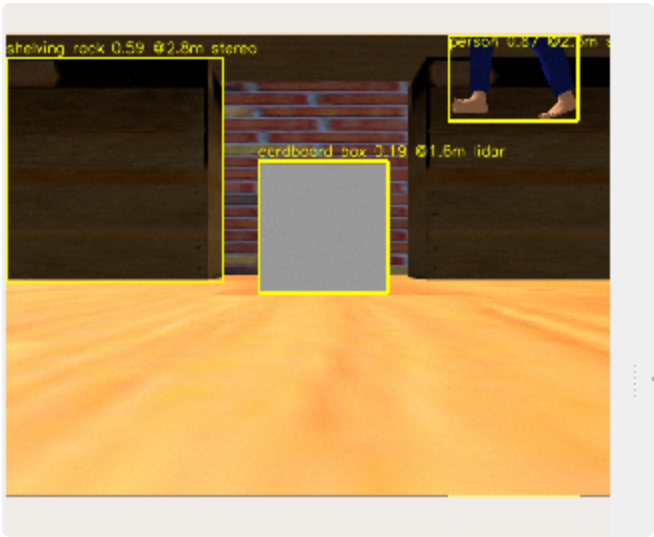


Figure: the camera view with YOLOv8-World detections. Each label shows the class, the confidence and the range, taken from stereo depth or the LiDAR (shelving rack, cardboard box, person).

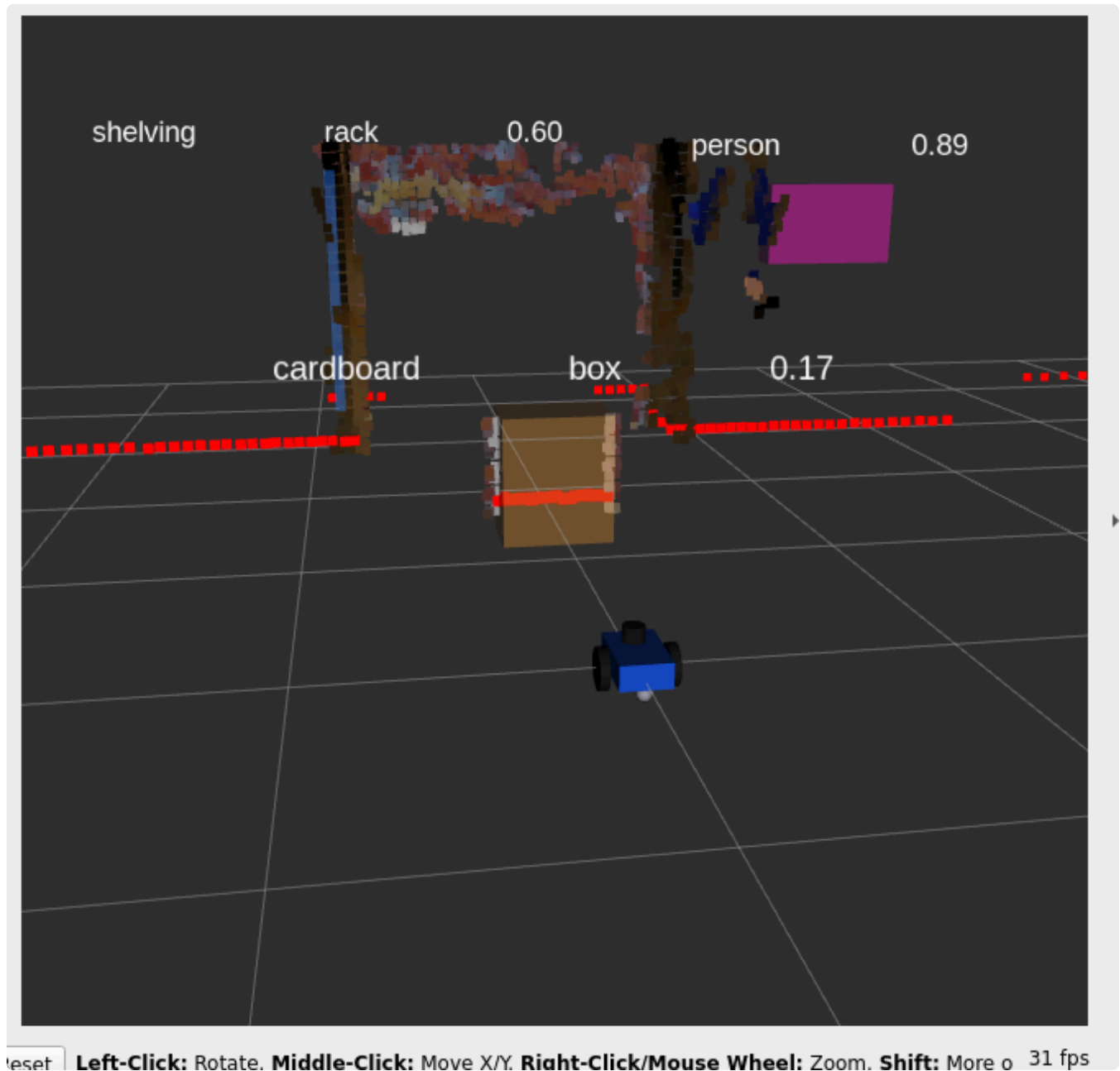


Figure: the same scene in RViz. Detections appear as labelled 3D markers over the point cloud, with the LiDAR scan in red (rack 0.60, box 0.17, person 0.89 confidence).

Why open-vocabulary. A custom detector would need a labelled warehouse dataset. The thesis work on this topic flagged small, class-imbalanced training data as a weakness, so this project avoids that dependency and accepts lower recall instead (6 of 10 objects found).

5. Object-based SLAM

Two mapping layers run together: a LiDAR occupancy map for free space, and a factor-graph object map for what the objects are and where they sit.

Occupancy map. `slam_toolbox` scan-matches the LiDAR and publishes `/map` and the robot pose on `/pose`. Its own `map -> odom` transform is switched off so that the object SLAM node can publish a better one.

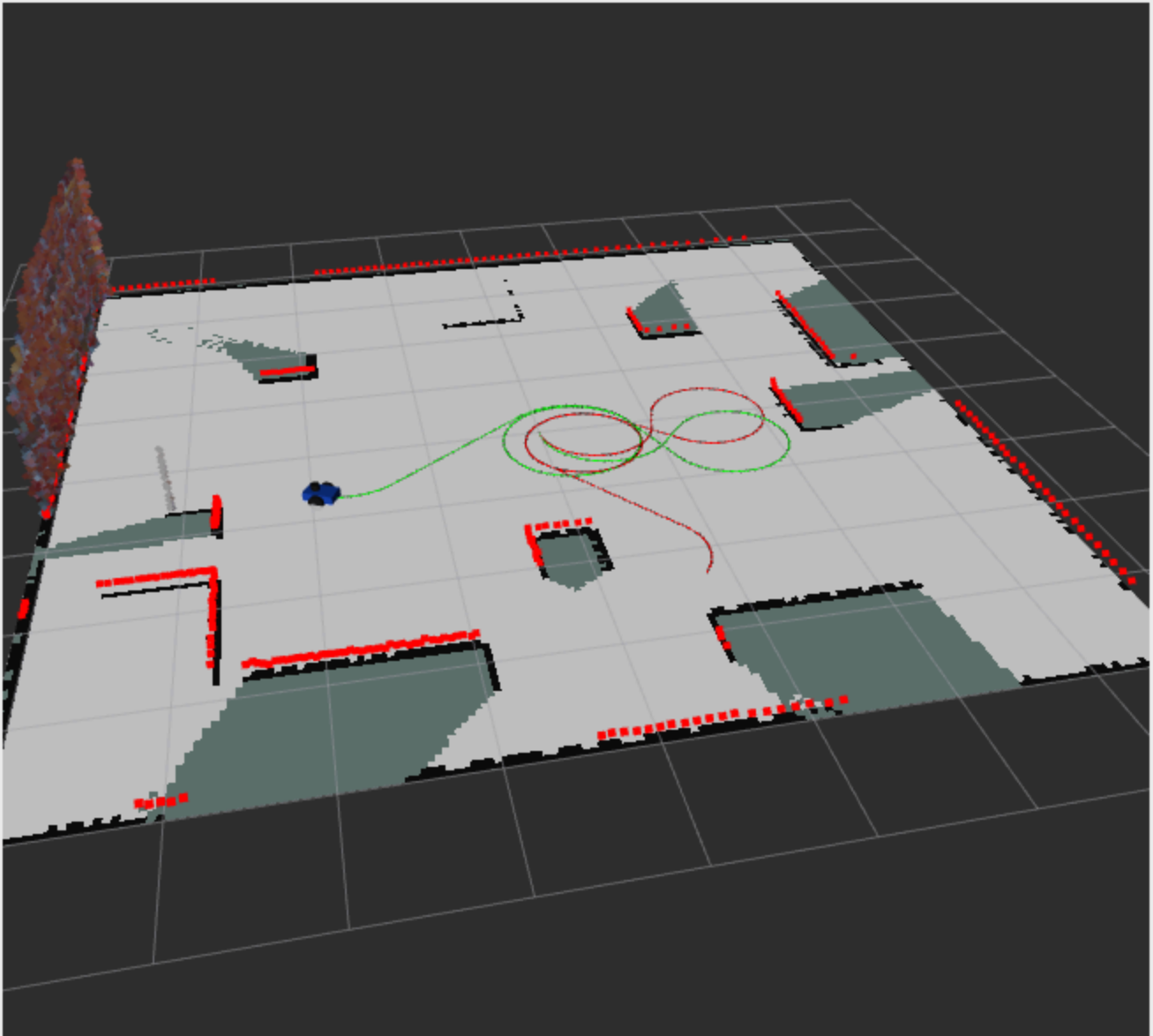


Figure: the `slam_toolbox` occupancy map during the mapping tour, with the robot's path drawn as coloured traces. Shaded regions are unexplored space behind obstacles.

Object map. `object_slam` builds a GTSAM iSAM2 factor graph. Every ~25 cm of driving it adds a keyframe, tied to the LiDAR pose and to the previous keyframe through odometry. Each YOLO sighting adds a bearing-and-range factor between the current keyframe and

an object. An object joins the map for good after three sightings from at least two keyframes. iSAM2 re-solves robot poses and object positions together, and the node publishes `map -> odom` as the best robot pose in the map composed with the inverse odometry pose.

Outputs. `/semantic_map/landmarks` (a `Detection3DArray` in the map frame), RViz markers, and a saved YAML object map. People are shown briefly in RViz but never stored as landmarks.

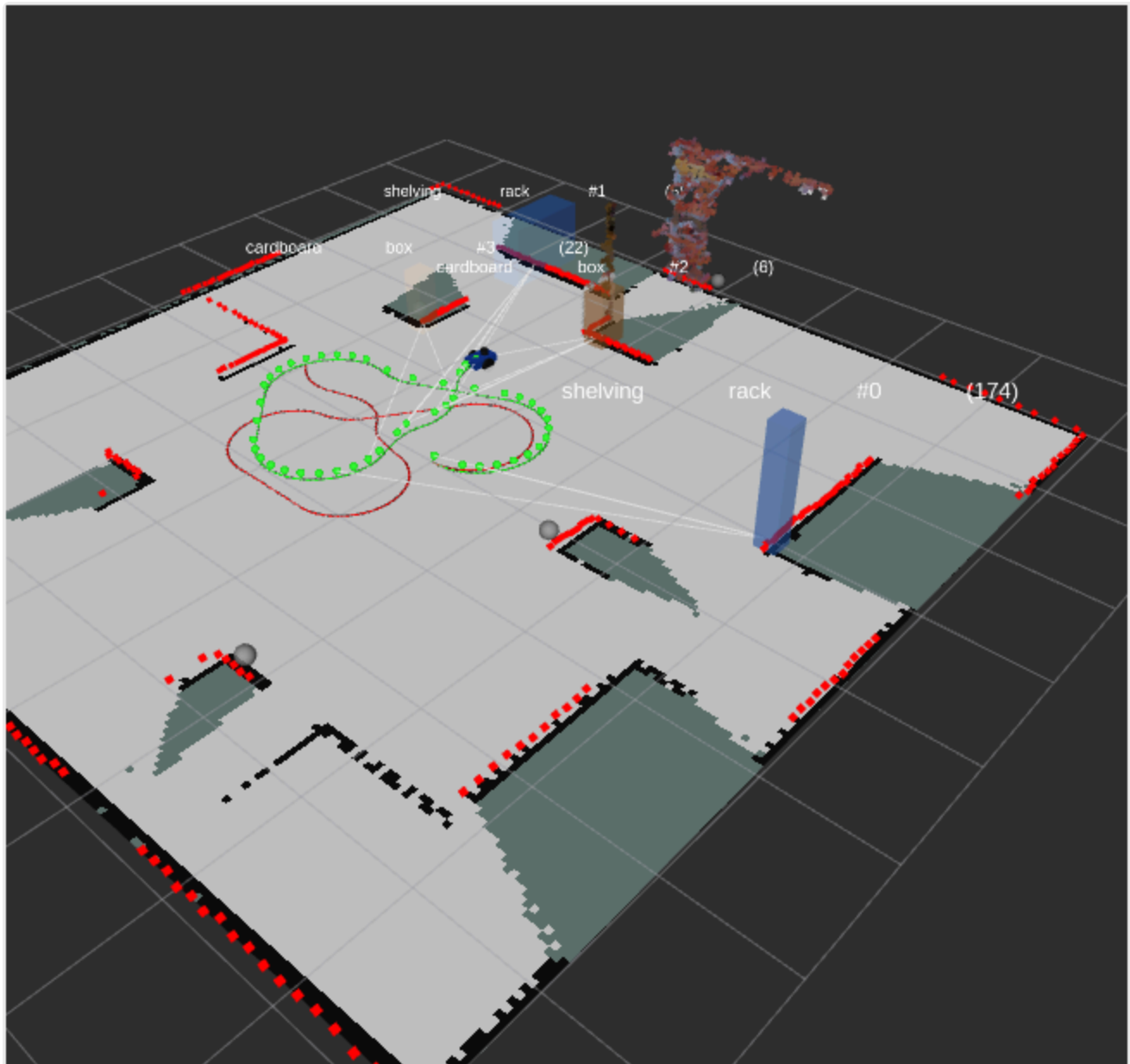


Figure: RViz view of the warehouse with the occupancy map, labelled object landmarks (shelving racks and cardboard boxes) and the robot's trajectory during the mapping tour.

Ghost-landmark filter. A YOLO detection with no LiDAR surface near it is probably a false positive. Before a landmark becomes a keep-out zone, the system requires a mapped LiDAR surface within 0.20 m (pallets are exempt because they can sit in free space). This stopped false boxes from blocking aisles, for example a cardboard box wrongly placed at (-1.14, 1.07) was ignored.

6. Nav2 configuration

Nav2 is configured so that the costmaps consume the semantic map and so that the robot fits through 0.65 m aisles.

Costmaps. Both the global and local costmaps stack three layers: a static layer from the SLAM `/map` , an obstacle layer fed by `/scan` and by the semantic point cloud, and an inflation layer.

Semantic costmap bridge. `semantic_costmap` converts every landmark into a keep-out disc sized by class, published as a `PointCloud2` that the obstacle layer reads:

Class	Keep-out radius	Notes
Rack	0.30 m	Persistent landmark
Pallet	0.25 m	Exempt from the ghost filter
Box	0.20 m	Needs LiDAR support within 0.20 m
Person	0.35 m	Expires after 1.5 s without a sighting

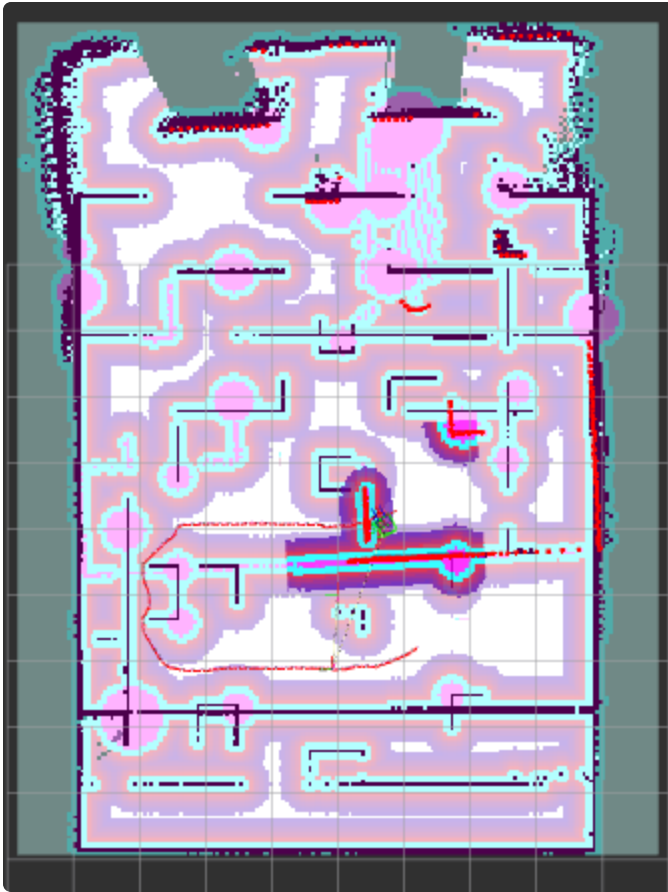
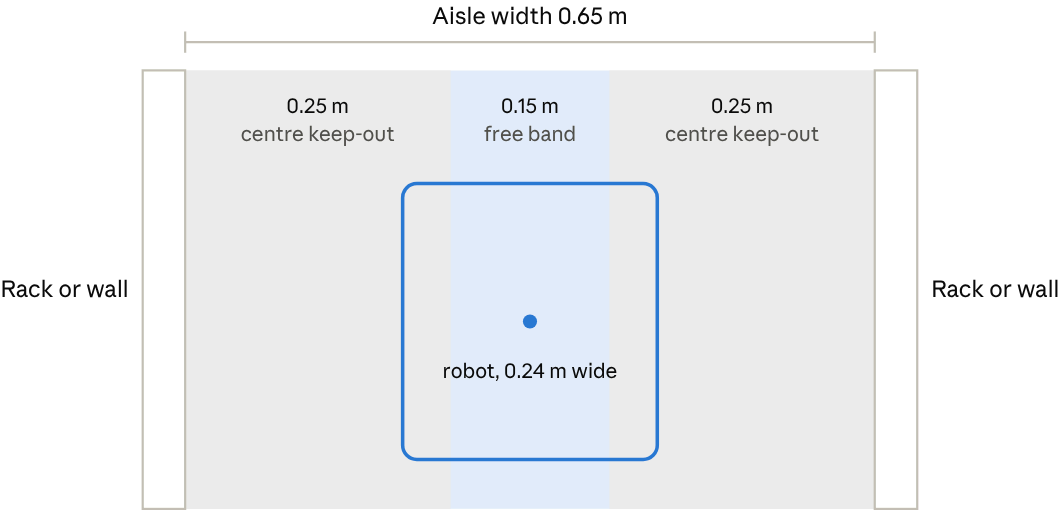


Figure: the global costmap in RViz. Walls and racks are surrounded by inflation bands, and the red line is the path the robot followed.

Narrow aisles. The footprint is an explicit polygon of $0.26\text{ m} \times 0.24\text{ m}$. The inscribed radius is 0.12 m , so the footprint check needs about 0.25 m of clearance from walls, leaving roughly 7.5 cm of slack in the 0.65 m aisle. Inflation is 0.45 m (scale 4.0) in the global costmap and 0.35 m (scale 5.0) in the local one. `tools/check_aisles.py` verifies offline that each aisle is passable under these settings.

A 0.65 m aisle leaves the robot centre only a 0.15 m band



aisle cross-section · top view, schematic

The drawing shows the aisle from above: the robot's centre may sit only in the 0.15 m band between the two keep-out zones.

Behaviour tree. A custom tree, `warehouse_nav.xml` , adds replanning and recovery behaviours. Each goal can name its own tree, which is how the planner and controller combinations are switched at run time.

Speed and tolerances. Maximum forward speed is 0.25 m/s, the goal position tolerance is 0.05 m, and the yaw tolerance is 0.30 rad.

7. Path planning

Three planner and controller combinations were implemented and are switched per goal through separate behaviour trees.

Configuration	Global planner	Local controller	Behaviour tree
astar_dwb	NavFn A*	DWB	<code>warehouse_nav.xml</code>
dijkstra_dwb	NavFn Dijkstra	DWB	<code>warehouse_nav_dijkstra.xml</code>
astar_mppi	NavFn A*	MPPI	<code>warehouse_nav_mppi.xml</code>

Global planner. NavFn runs on the global costmap. A* adds a heuristic to Dijkstra's search, so on a small warehouse grid both expand a similar region and return near-identical paths.

Local controller. DWB samples velocity commands and scores each trajectory with critics, including an obstacle-footprint critic that checks the whole robot outline. MPPI samples 1000 trajectories of 56 steps (0.05 s each) with a cost critic that also considers the footprint, limits of 0.25 m/s forward and 1.0 rad/s turning.

Why not TEB. The task brief names TEB as an example local planner. TEB has no apt package on Humble and would have to be built from source, so MPPI, which ships with Humble's Nav2, was chosen as the second controller.

Dynamic obstacles. A walking person is simulated by an invisible collision cylinder driven by `worker_walker` at 0.6 m/s, so the LiDAR sees it. The local costmap updates at 10 Hz and the semantic costmap adds a 0.35 m keep-out disc around the person. The controllers react to the person's current position but do not predict its motion.

8. Validation methodology

The Gazebo world file is the ground truth, and `tools/validate_pipeline.py` scores the pipeline automatically in four stages.

1. **Mapping tour.** The robot drives 11 waypoints through the warehouse while SLAM and the object map build. The SLAM map is then scored against the world: wall recall, rack recall and precision, with a 0.10 m tolerance.
2. **Semantic map.** Each landmark is checked against the true object footprint for position and class.
3. **Mission.** The robot navigates 7 waypoints, one per aisle or area. Each leg records status, time, path length, final error, planning time (measured with `perf_counter`), the closest LiDAR return and the minimum separation.
4. **Dynamic legs.** The robot navigates while the simulated person walks across its route, and the minimum separation from the person's true position is recorded.

The script prints a PASS/FAIL table against fixed targets and writes JSON and Markdown reports to `reports/`:

Metric	Target
Wall recall	≥ 0.85
Rack recall	≥ 0.40
Map precision	≥ 0.85

Metric	Target
Landmarks in correct footprint	≥ 0.75
Racks found	≥ 2
Mission success rate	≥ 0.90
Final goal error	≤ 15 cm
Closest LiDAR clearance	≥ 0.16 m
Dynamic-leg success rate	≥ 0.75
Person separation	≥ 0.45 m

On a failed leg the script prints a 4 m $\times$ 4 m picture of the global costmap around the robot, which is how the aisle-entrance stalls were diagnosed. Offline unit tests (`test_semantic_ops.py` , `test_validation_ops.py`) cover the geometry code without the simulator.

9. Results

The SLAM map misses its wall target by 0.6 points, and A* + DWB is the only configuration that completes the mission (6 of 7 waypoints).

Map precision is 99.9%, but wall recall misses its target by 0.6 points

Share of true world cells recovered, 0.10 m tolerance. Tick = validation target.



`validate_pipeline.py`, `astar_dwb run` · SLAM map vs Gazebo world file

The semantic map holds 7 landmarks, all within 0.5 m of a real object with the correct class, covering 6 of the 10 objects in the world (2 of the 3 racks). Not found: shelf_3, box_5, pallet_1 and pallet_2. The north wall was only 37.5% mapped, which pulls wall recall down

to 84.4%. Navigation results by configuration:

Configuration	Mission legs reached	Mean time	Mean path	Mean / max error	Dynamic legs reached	Closest to person
A* + DWB	6 of 7	20.4 s	3.4 m	4.7 / 5.8 cm	4 of 4	0.37 m
Dijkstra + DWB	2 of 7	26.8 s	4.3 m	7.2 / 9.7 cm	4 of 4	0.37 m
A* + MPPI	0 of 7	n/a	n/a	n/a	0 of 4	1.93 m (never reached the person's route)

Scored on the best combination (A* + DWB), the validation targets come out as follows:

Target	Measured	Required	Result
Wall recall	0.84	≥ 0.85	Miss
Rack recall	0.80	≥ 0.40	Pass
Map precision	1.00	≥ 0.85	Pass
Landmarks near a real object	1.00	≥ 0.75	Pass
Racks found (of 3)	2	≥ 2	Pass
Mission legs reached	0.86	≥ 0.90	Miss
Worst final error	5.8 cm	≤ 15 cm	Pass
Closest LiDAR return	0.13 m	≥ 0.16 m	Miss
Dynamic legs reached	1.00	≥ 0.75	Pass
Closest distance to person	0.37 m	≥ 0.45 m	Miss

The global planners were also compared without driving, over 7 paths: A* averaged 3.89 m in 43.4 ms and Dijkstra 3.85 m in 48.1 ms. Dijkstra's paths are 4 cm shorter and A* is 5 ms faster, a difference too small to matter at this map size. Section 10 explains the Dijkstra and MPPI failures.

10. Limitations and future work

Four targets were missed in the final run, and the largest gap is that only A* + DWB navigates the whole warehouse.

Stalls at the east aisle. The goal at (3.58, 1.60) timed out after 90 s in all three configurations, with Nav2 reporting "Failed to make progress". The costmap pictures show free space around the robot and a valid plan, so the planner is not the cause; the controller stalls with only about 7 cm of footprint slack in the aisle. A* + DWB recovered and completed the other six legs.

Dijkstra + DWB. After reaching (3.58, -1.60) it aborted waypoints 4 to 7 with the robot stopped near (3.3, -1.47) inside the aisle, so one stall cascaded into four failed legs. The same planner worked on all four dynamic legs, so the cause is the controller and aisle geometry, not the path.

A* + MPPI. It reached 0 of 7 waypoints and 0 of 4 dynamic legs. It drove toward most goals and stopped 24-45 cm short, with small reversing and turning commands at the end, so it neither met the 5 cm goal tolerance nor cleared the aisle. The likely cause is that the MPPI critics and noise were not tuned for a 5 cm tolerance in 0.65 m aisles; goal and path critics would be tuned first.

Person separation. Minimum distance to the walking person was 0.37 m (touching is 0.37-0.43 m) against a 0.45 m target, although all four dynamic legs reached the goal. DWB has no motion prediction and the person walks at 0.6 m/s against the robot's 0.25 m/s. A Nav2 collision monitor with speed limiting near people, or a predictive controller such as TEB, would address this.

Detection and mapping coverage. 6 of 10 objects were found; shelf_3, box_5, pallet_1 and pallet_2 were not. The north wall was only 37.5% mapped, which is why wall recall is 0.84 against 0.85. A tour waypoint near the north wall and more views of the unseen objects would fix both.

Scope. Everything was validated in simulation only, in one full run of all three configurations (2233 s of wall time). The Gazebo actor has no collision body, so a separate invisible cylinder stands in for the person. TEB was not built.

Future work: an approach waypoint and heading alignment at each aisle, MPPI tuning for the tight aisles and 5 cm tolerance, a collision monitor, TEB from source, a wider mapping tour, and hardware testing.

11. Conclusion and reproduction

The project shows a working path from raw wheel ticks to autonomous navigation in a tight warehouse, with a semantic map that is measured against ground truth rather than assumed.

To reproduce it:

1. Clone `github.com/dikshajangam26/autonomous-diffbot-ros2`, run `docker compose build` and `docker compose up -d`, then enter the container and run `colcon build -s symlink-install`.
2. Terminal 1: `ros2 launch task_robotics sensors_sim.launch.py object_slam:=true people:=true gui:=false rviz:=false`
3. Terminal 2: `ros2 launch task_robotics nav2.launch.py`, and wait for "Managed nodes are active".
4. Terminal 3: `python3 tools/validate_pipeline.py`, or `python3 tools/nav_goal_test.py 3.575 1.6` for a single goal.

A full validation of all three configurations took about 37 minutes of wall time (2233 s); one configuration alone takes roughly a third of that.